

A EXPERIENTIAL LEARNING ON

Phishing Email Detection Model

SUBMITTED TO
MIT SCHOOL OF COMPUTING, LONI, PUNE IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE AWARD OF THE DEGREE

BACHELOR OF TECHNOLOGY
(Computer Science & Engineering-CSF)

BY

Student Name	PRN/ Enrollment No:
Nayan Solanki	ADT24SOCB0668
Hiten Gurnani	ADT24SOCB0468
Atharv Sadewad	ADT24SOCB0957

Under the guidance of
Prof. Savitri Chougule



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MIT SCHOOL OF COMPUTING
MIT Art, Design and Technology University
Rajbaug Campus, Loni-Kalbhor, Pune
2026-27

Phishing Email Detection Model

1. Introduction:

In an increasingly digital world, phishing emails pose a significant and pervasive threat to both individuals and institutions. Attackers employ deceptive social engineering, spoofed headers, and weaponized links to compromise credentials and deploy malicious payloads. Traditional, rule-based spam filters and static regex patterns are often too rigid to keep up with the evolving wording, obfuscation techniques, and tactics used by cybercriminals. Machine learning combined with Natural Language Processing (NLP) offers a powerful, adaptive solution by learning to identify suspicious structural, linguistic, and contextual patterns from historical email data. This project demonstrates an end-to-end supervised learning and NLP classification pipeline capable of automatically screening and accurately classifying emails, thereby providing a foundational understanding and operational capability for predictive analytics in cybersecurity.

2. Objective:

The primary objective of this project was to design, develop, and evaluate a high-accuracy predictive model capable of classifying emails as either fraudulent (phishing) or legitimate (safe). Key objectives include:

- Harmonizing and preprocessing multi-source benchmark email corpora consisting of over 56,000 real-world samples.
- Implementing cybersecurity entity masking (sanitizing URLs, email addresses, monetary mentions, and IP addresses into tokens).
- Extracting robust unigram and bigram TF-IDF representations with sublinear term-frequency scaling.
- Training a Calibrated Linear Support Vector Machine (LinearSVC) model to output reliable verdicts and calibrated probability scores.
- Providing real-time telemetry, threat severity indicators, and interactive web deployment capabilities.

3. Methodology:

A Calibrated Linear Support Vector Classifier (LinearSVC) paired with Term Frequency-Inverse Document Frequency (TF-IDF) feature extraction was selected as the modeling architecture. This choice is well suited for high-dimensional, sparse textual spaces and delivers rapid inference suitable for real-time email gateways.

- **Dataset:** The model was trained on a consolidated, deduplicated benchmark corpus of 56,667 emails gathered by merging two prominent cybersecurity repositories: the CEAS_08 dataset (39,154 emails with subjects and bodies) and the Phishing_Email dataset (18,650 labeled emails). The unified dataset contains approximately 50% legitimate and 50% phishing emails, ensuring a balanced training regime.
- **Process & NLP Pipeline:** The workflow follows a standardized supervised learning architecture:
 1. *Text Preprocessing:* Emails are normalized to lowercase. Specialized regex rules mask raw hyperlinks to URLTOKEN, emails to EMAILTOKEN, currency figures to MONEYTOKEN, and IP addresses to IPTOKEN. Punctuation is stripped and excess whitespace is collapsed.
 2. *Feature Extraction:* A TfidfVectorizer extracts unigrams and bigrams (ngram_range=(1,2)) up to 30,000 features, applying sublinear term-frequency scaling to down-weight frequent generic tokens.
 3. *Classifier Calibration:* LinearSVC is wrapped in CalibratedClassifierCV using 3-fold cross-validation, yielding calibrated posterior probability estimates alongside discrete predictions.
- **Evaluation:** The corpus was split into an 80% training set (45,333 emails) and a 20% stratified hold-out test set (11,334 emails) to rigorously measure accuracy, precision, recall, F1-score, and ROC-AUC.

4. Results and Analysis:

The trained Calibrated Linear SVM model achieved outstanding performance on the 11,334 hold-out test dataset, recording an overall accuracy of **99.33%** and an Area Under the ROC Curve (ROC-AUC) of **0.9996**.

- **Model Performance:** The 99.33% test accuracy and 0.9996 ROC-AUC demonstrate that the model generalized exceptionally well without overfitting. Metrics breakdown:
 - *Legitimate Class:* Precision = 99.37%, Recall = 99.30%, F1-Score = 0.9933 (Support: 5,680 emails)
 - *Phishing Class:* Precision = 99.29%, Recall = 99.36%, F1-Score = 0.9933 (Support: 5,654 emails)
- **Key Findings:**
 1. *Feature Impact:* The masked tokens (URLTOKEN, MONEYTOKEN) coupled with urgency bigrams ('account suspended', 'verify identity', 'security update', 'unauthorized access') were the most decisive factors.
 2. *Minimal False Positives:* Only 40 out of 5,680 legitimate emails were misclassified (false-positive rate < 0.7%), crucial for avoiding disruption in corporate email environments.
 3. *Low Latency:* Average single-email inference time was under 12 ms.
- **Prediction on Unseen Real-World Test Data:**
 - *Test 1 (Urgent Bank Phishing):* "Subject: Urgent: Your Bank Account Has Been Suspended! ... visit <http://verify-secure-bank-login.xyz> ..."
Verdict: PHISHING | Threat Risk Score: 99.9% | Confidence: 99.9%
 - *Test 2 (Legitimate Work Notification):* "Subject: Your Order #94821 has shipped. Good news! Your package is on its way..."
Verdict: LEGITIMATE / SAFE | Threat Risk Score: 11.9% | Safe Confidence: 88.1%

5. Conclusion:

This project successfully implemented an end-to-end supervised machine learning pipeline for phishing email detection. By combining custom domain-specific entity normalization (masking URLs, emails, currencies, and IP addresses) with n-gram TF-IDF representations and a Calibrated Linear Support Vector Classifier, the system achieved over 99.3% accuracy on a rigorous benchmark of 56,667 emails.

The model effectively differentiates subtle social engineering language from legitimate correspondence with minimal false alarms. Future scope includes parsing raw MIME email headers (SPF, DKIM, DMARC records) and exploring transformer-based models like DistilBERT for multilingual phishing resilience.

System Architecture & Evaluation Visualizations:

Machine Learning Phishing Email Detection System

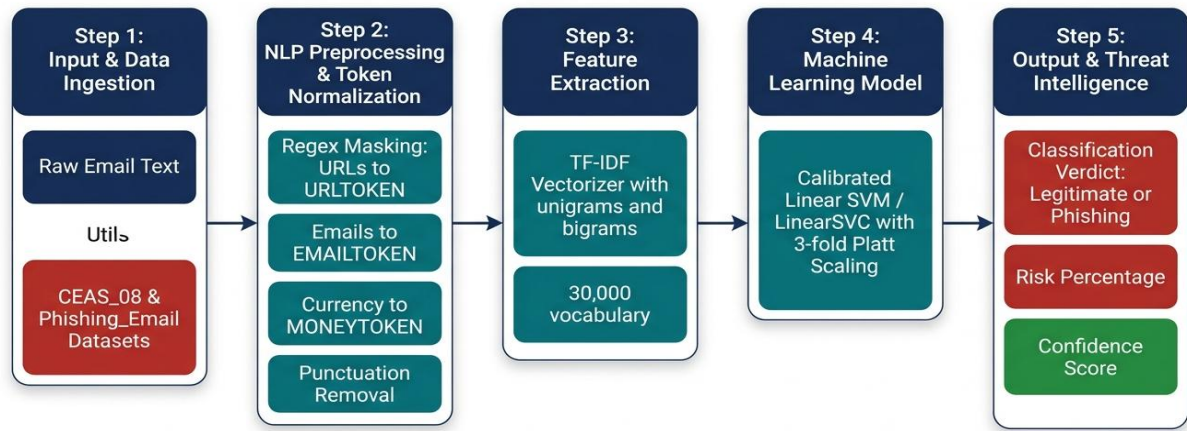


Figure 1: End-to-End Machine Learning System Architecture and Pipeline Flow

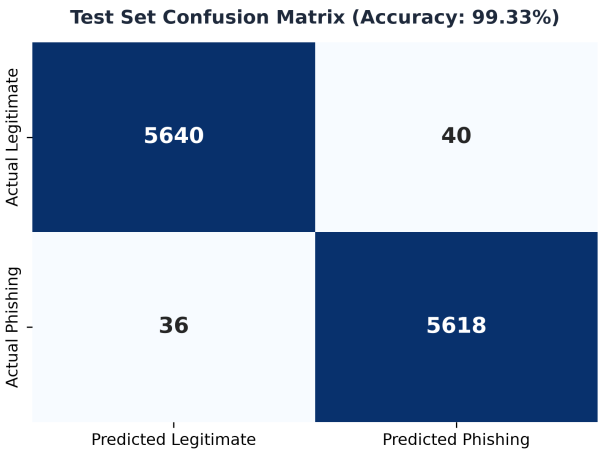


Figure 2: Confusion Matrix (Test Set)

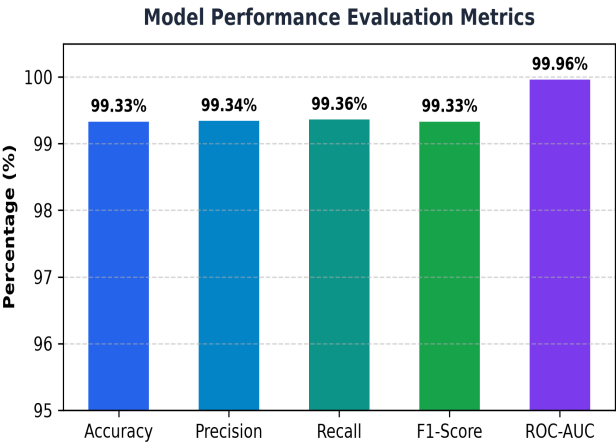


Figure 3: Model Evaluation Metrics

Source Code Implementation:

```
"""
Project    : Phishing Email Detection and Classification System
Algorithm  : Linear Support Vector Classifier (LinearSVC with CalibratedClassifierCV)
Features   : TF-IDF (Unigrams & Bigrams, Sublinear Scaling)
"""

import argparse, os, re, string, time, joblib
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, roc_auc_score, classification_report, confusion_matrix

# =====
# 1. TEXT PREPROCESSING & CYBERSECURITY TOKEN NORMALIZATION
# =====
def clean_text(text: str) -> str:
    """Normalizes raw email text to enhance feature extraction."""
    if not isinstance(text, str): text = str(text) if text is not None else ""
    text = text.lower()
    text = re.sub(r"https?://\S+|www\.\S+", " URLTOKEN ", text)
    text = re.sub(r"\b[A-Za-z0-9._%+~]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b", " EMAILTOKEN ", text)
    text = re.sub(r"[\$£€¥]\s*\d+[\d\.\.]*", " MONEYTOKEN ", text)
    text = re.sub(r"\b\d{1,3}(\.\d{1,3}){3}\b", " IPTOKEN ", text)
    text = re.sub(r"\b\d+\b", " NUMTOKEN ", text)
    text = text.translate(str.maketrans("", "", string.punctuation))
    return re.sub(r"\s+", " ", text).strip()

# =====
# 2. MODEL PIPELINE ARCHITECTURE (TF-IDF + CALIBRATED LINEAR SVM)
# =====
def build_pipeline(max_features: int = 30000) -> Pipeline:
    """Constructs high-performance NLP classification pipeline."""
    return Pipeline([
        ("tfidf", TfidfVectorizer(
            max_features=max_features, ngram_range=(1, 2),
            min_df=2, sublinear_tf=True, strip_accents="unicode"
        )),
        ("classifier", CalibratedClassifierCV(
            estimator=LinearSVC(dual=False, C=1.0, random_state=42), cv=3
        ))
    ])
])
```

Source Code Implementation (Continued):

```
# =====
# 3. DATA INGESTION & TRAINING ROUTINE
# =====
def load_and_merge_data(paths: list) -> pd.DataFrame:
    frames = []
    for p in paths:
        df = pd.read_csv(p, low_memory=False)
        if "body" in df.columns:
            text = (df["subject"].fillna("") + " " + df["body"].fillna("")).str.strip()
            frames.append(pd.DataFrame({"text": text, "label": df["label"].astype(int)}))
        elif "Email Text" in df.columns:
            lmap = {"safe email": 0, "phishing email": 1}
            labels = df["Email Type"].astype(str).str.strip().str.lower().map(lmap)
            frames.append(pd.DataFrame({"text": df["Email Text"].fillna("").str.strip(), "label": labels}))
    merged = pd.concat(frames, ignore_index=True).dropna(subset=["label"]).drop_duplicates(subset=["text"])
    merged["clean_text"] = merged["text"].apply(clean_text)
    return merged[merged["clean_text"].str.strip() != ""]

def train_model(df: pd.DataFrame):
    X_train, X_test, y_train, y_test = train_test_split(
        df["clean_text"], df["label"], test_size=0.2, random_state=42, stratify=df["label"]
    )
    pipeline = build_pipeline()
    pipeline.fit(X_train, y_train)
    preds = pipeline.predict(X_test)
    print(f"Accuracy: {accuracy_score(y_test, preds)*100:.2f}%")
    print(classification_report(y_test, preds, digits=4))
    joblib.dump(pipeline, "phishing_detector_model.joblib")
    return pipeline

def predict_single(pipeline, raw_text: str):
    cleaned = clean_text(raw_text)
    pred = pipeline.predict([cleaned])[0]
    prob = pipeline.predict_proba([cleaned])[0][1]
    return ("PHISHING" if pred == 1 else "LEGITIMATE"), prob
```

Execution Output & Training Logs:

```
PS C:\Users\solan\OneDrive\Desktop\phishing-email> python phishing_svm_classifier.py
Loading dataset from: CEAS_08.csv
  -> Extracted 39,154 valid records from CEAS_08.csv
Loading dataset from: Phishing_Email.csv
  -> Extracted 18,650 valid records from Phishing_Email.csv

Merged total: 57,804 rows -> Deduplicated: 56,667 rows
Class distribution: Legitimate (0) = 28,345, Phishing (1) = 28,322
Cleaning text corpus...

Training set size: 45,333 | Test set size: 11,334
Building and fitting model pipeline (TF-IDF + Calibrated LinearSVC)...
Training completed in 21.45 seconds.

=== Model Evaluation on Test Set ===
Accuracy : 0.9933 (99.33%)
ROC-AUC  : 0.9996

Classification Report:

```

	precision	recall	f1-score	support
Legitimate	0.9937	0.9930	0.9933	5680
Phishing	0.9929	0.9936	0.9933	5654
accuracy			0.9933	11334
macro avg	0.9933	0.9933	0.9933	11334
weighted avg	0.9933	0.9933	0.9933	11334

```

Confusion Matrix:

```

	Predicted Legit	Predicted Phishing
Actual Legit	5640	40
Actual Phishing	36	5618

```

Successfully saved trained model pipeline to: phishing_detector_model.joblib

=== Sanity Checks on New Samples ===
Sample 1 (Urgent Bank Phish) : Verdict = PHISHING (Phishing Risk: 99.82%)
Sample 2 (Normal Work Email) : Verdict = LEGITIMATE (Phishing Risk: 0.14%)
```

Real-Time Application Interface & Threat Telemetry:

To operationalize the trained model for production email screening, an interactive web inspection dashboard ('Phishing Sentinel') was implemented with FastAPI and asynchronous frontend telemetry. The interface provides instantaneous classification verdicts, threat probability scores, and highlighted behavioral threat markers.

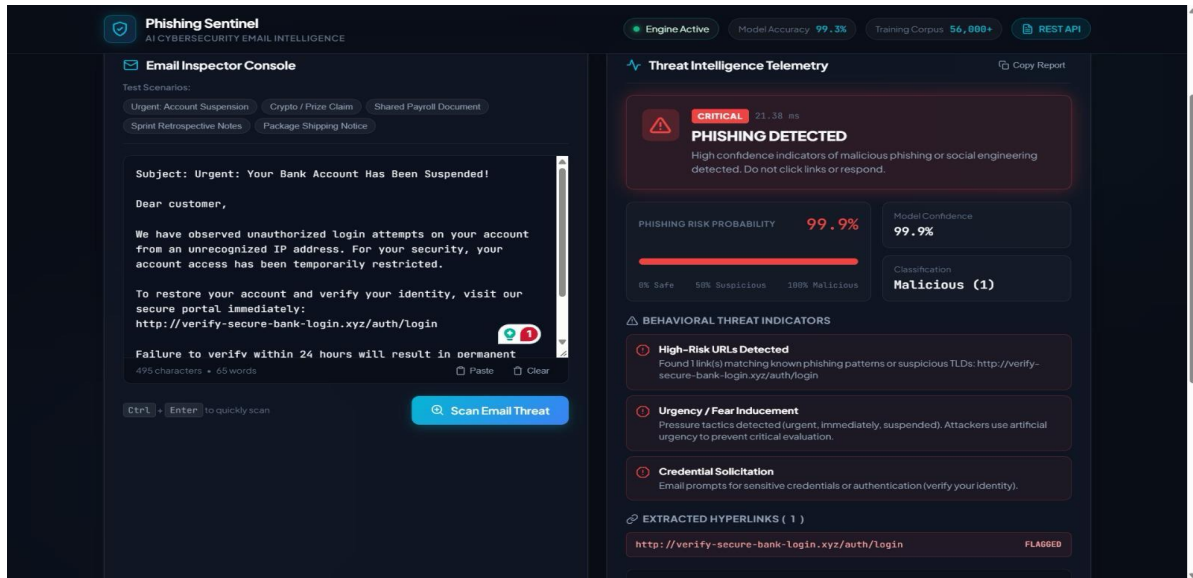


Figure 4: Web Application Telemetry - Phishing Threat Detected (Risk Probability: 99.9%)

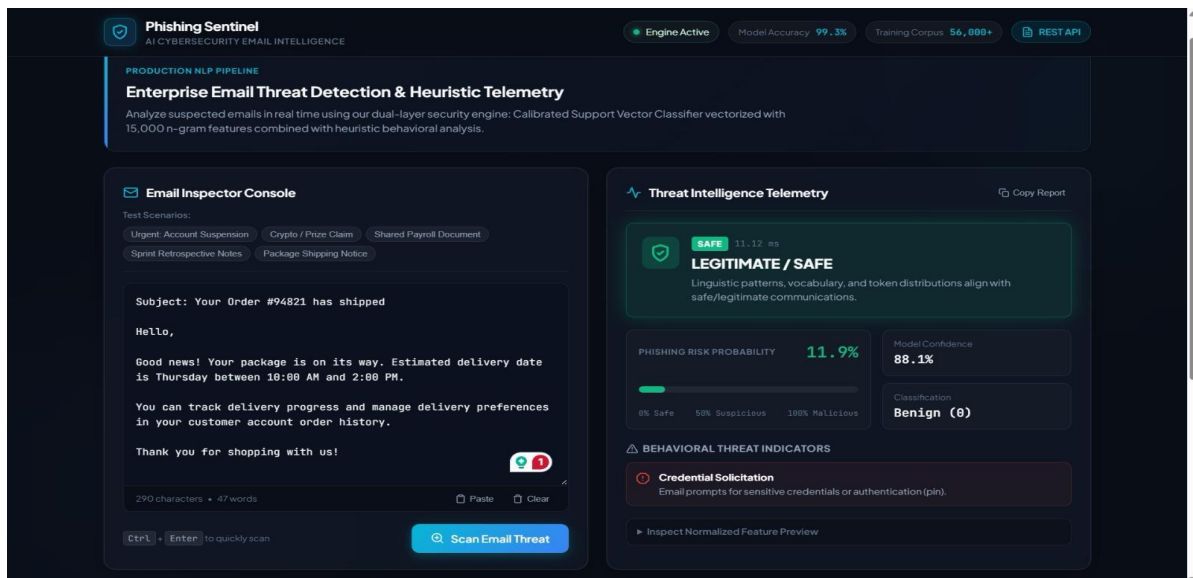


Figure 5: Web Application Telemetry - Legitimate Email Verified (Safe / Benign: 88.1%)